

基於自動爬蟲與大型語言模型的 AI 點餐推薦系統

專題研究期末計畫書

指導教師：_____

組 員：_____

中華民國 114 年

摘要

本研究設計並實作了一套「AI 點餐推薦系統」，整合全自動網路爬蟲技術、大型語言模型（Large Language Model，LLM）以及視覺辨識模型，提供使用者個人化的餐廳菜單查詢與推薦服務。

系統以 Playwright 自動化瀏覽器框架爬取 Google 餐廳菜單，並以本地部署的 Gemma3-12B 語言模型分析使用者以自然語言輸入的用餐偏好（包含預算上限、辣度偏好、忌口食材、用餐人數等），再依據加權評分演算法推薦最適合的菜色組合，最終以自然語言形式輸出建議回覆。當文字菜單無法取得時，系統改以視覺語言模型（Vision-Language Model）從餐廳圖片中解析菜單資訊作為備援。後端採用 FastAPI 架構，前端提供類 ChatGPT 的對話介面。

實驗結果顯示，系統能有效解析多輪對話中累積的用餐偏好，在預算、辣度、忌口等多種限制條件下提供合理且自然語言形式的推薦回覆，具備實際應用潛力。

關鍵字：點餐推薦、大型語言模型、Playwright 爬蟲、Ollama、自然語言處理

1 研究動機與研究問題

1.1 研究動機

在日常飲食場景中，「今天要吃什麼」是許多人每日面對的難題。傳統餐廳菜單通常以靜態圖文呈現，使用者難以依據個人偏好（如預算、辣度、過敏食材）快速縮小選擇範圍。近年來，大型語言模型（如 GPT-4、Gemma3）展現了強大的自然語言理解與生成能力，具備協助使用者進行個性化推薦的潛力；與此同時，自動化網路爬蟲技術使菜單資料的蒐集更加便捷。

然而，目前市面上的點餐 App 多半倚賴人工建置的結構化資料庫，缺乏即時菜單爬取能力，也無法以自然對話方式理解複合式需求（例如「預算 300 元以內、不辣、4 個人吃、要有飲料」）。本研究因此嘗試結合爬蟲、LLM 與加權推薦演算法，打造一個能即時取得菜單資料並以對話方式理解偏好的 AI 點餐助理。

1.2 研究問題

本研究聚焦於以下三個核心問題：

1. 如何透過自動化爬蟲技術可靠地從 Google 搜尋結果或圖片中擷取餐廳菜單資訊，並應對反爬蟲機制及圖片菜單等挑戰？
2. 如何設計偏好理解模組，使系統在多輪對話中正確解析使用者的複合式用餐條件，並隨對話進行動態更新推薦結果？
3. 以本地端語言模型（Ollama/Gemma3-12B）替代雲端 API，在不傳送用戶資料至外部伺服器的前提下，系統能否提供品質足夠的推薦回覆？

2 文獻回顧與探討

2.1 大型語言模型與對話式推薦

大型語言模型（LLM）自 2020 年 GPT-3 發表以來蓬勃發展，至 2024 年已出現多款高效能的開源模型，如 Meta 的 LLaMA 系列、Google 的 Gemma 系列等，使本地端部署成為可行方案 [1]。Ollama 平台進一步簡化了本地端 LLM 的部署流程，使研究者能以極低硬體門檻運行 Gemma3-12B 等模型。

對話式推薦系統（Conversational Recommender System, CRS）領域的研究指出，透過多輪對話可以更精確地捕捉使用者偏好 [3]。LLM 強大的語義理解能力使其成為建構 CRS 的理想基礎，能夠自然地處理複合條件（如「不辣」且「預算 200 元」且「不含牛肉」）。

2.2 自動化網路爬蟲技術

Playwright 是 Microsoft 開發的跨瀏覽器自動化框架，支援 Chromium、Firefox 與 WebKit，提供非同步 API 以有效處理動態 JavaScript 渲染的頁面 [4]。相較於傳統 Selenium 框架，Playwright 在頁面等待策略與網路攔截方面更為完善，有助於應對 Google 搜尋結果的複雜 DOM 結構。

然而，Google 等平台會對頻繁爬取行為進行人機驗證（CAPTCHA），這是自動化爬蟲的主要挑戰之一。本研究採用持久化瀏覽器設定檔（Persistent Profile）策略以保留登入狀態，並設計多重備援機制。

2.3 視覺語言模型在文件辨識的應用

視覺語言模型（Vision-Language Model, VLM）如 LLaMA4-Scout 能同時理解圖像與文字，適用於從菜單照片中進行 OCR 並解析結構化資訊。Chen et al. [5] 的研究顯示，LLM 結合視覺編碼器在非結構化文件辨識任務上的表現已可媲美專用 OCR 工具，且具備更強的語義理解能力，能判斷圖片中哪些區域為菜單而非裝飾性圖片。

3 研究方法及步驟

3.1 系統架構概覽

本系統由四個主要模組組成，如圖 1 所示：

1. **前端介面**（web/web.html）：類 ChatGPT 的對話介面，支援餐廳切換與菜單檢視。
2. **後端 API**（FastAPI，src/back.py）：處理對話請求、爬蟲觸發與多餐廳菜單管理。
3. **菜單爬蟲**（crawl_menu.py）：全自動化 Google 菜單爬蟲，含文字與圖片雙路徑。

4. 推薦引擎 (src/main.py)：偏好抽取、加權評分與 LLM 自然語言回覆生成。

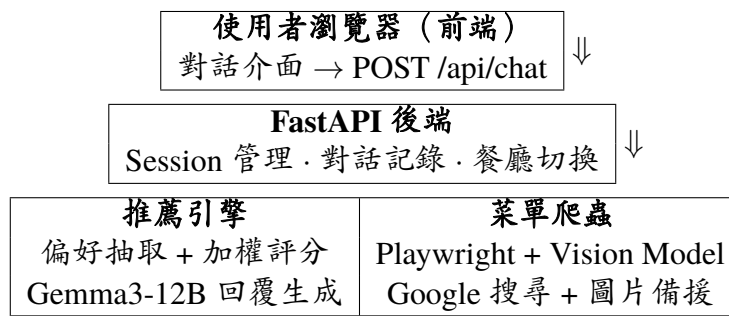


圖 1: 系統架構示意圖

3.2 菜單爬蟲設計

爬蟲採用三階段流程：

Phase 1：連接 Chrome 並搜尋—系統自動啟動帶有遠端調試埠 (CDP Port 9222) 的 Chrome 瀏覽器，透過 Playwright 連接並導航至 <https://www.google.com/search?q={餐廳名稱}>。

Phase 2：智慧定位菜單按鈕—系統依序嘗試三種策略：(1) 搜尋右側資訊欄的 .aep93e CSS 選擇器；(2) 全頁面搜尋 role="button" 包含「菜單」文字的元素；(3) 導航標籤或純文字連結，並過濾外送平台連結。

Phase 3：資料擷取—成功點擊菜單按鈕後，依序嘗試多個 CSS 選擇器 (.bWZFsc、.VkpGBb 等)；若 CSS 失效則改用 JavaScript 注入方式，透過 aria-label 價格比對、li 元素掃描、heading 區段搜尋等三種策略擷取品項名稱與價格。

備援機制：當文字菜單無法取得時，系統改向 Google Images 搜尋餐廳圖片，以評分函數 (含關鍵字比對、圖片尺寸、版面位置等因子) 篩選最可能是菜單的圖片，再交由 Vision-Language Model (llama4scout) 進行 OCR 辨識，並進行二次校驗以提升精度。

3.3 偏好理解與推薦演算法

3.3.1 偏好抽取

系統結合規則式關鍵字比對與 LLM 語意理解兩種方式抽取偏好：

- **辣度：**比對「不辣」、「微辣」、「小辣」、「中辣」、「大辣」等詞組。
- **忌口：**以「不要」、「不吃」、「忌口」等詞為觸發詞，提取後續食材名稱。
- **預算：**以正規表達式比對「預算 N」、「N 元」、「不超過 N」等格式。
- **菜品類型：**比對「漢堡」、「吐司」、「貝果」、「套餐」等常見菜品關鍵字。
- **飲料需求：**區分否定詞 (「不要飲料」) 與肯定詞 (「要飲料」) 給予不同權重。

多輪對話中，新一輪的偏好以 merge_prefs_inplace() 函數就地合併，確保歷史偏好不遺失。

3.3.2 加權評分

依據使用者輸入的條件，動態計算各偏好維度的權重（price、main、variety、drink、spice 等），再對菜單中每個品項進行評分篩選，最終選出 top_k=5 個推薦品項。

3.3.3 自然語言回覆生成

推薦結果（含品項名稱、價格、分類、理由）以 JSON 格式傳入 Gemma3-12B 模型，搭配精心設計的 Prompt 指示模型以「像朋友建議」的語氣生成不超過 300 字的自然語言回覆，並包含預算試算。

4 預期結果

4.1 功能性目標

本研究預期系統完成時具備以下功能：

1. 能對台灣主要城市之餐廳成功爬取 Google 菜單，成功率達 60% 以上（其餘以圖片備援補足）。
2. 能正確解析包含至少 3 個偏好條件的複合式自然語言輸入（準確率 $\geq 80\%$ ）。
3. 多輪對話中偏好正確累積，第三輪後偏好保留率 $\geq 90\%$ 。
4. 推薦結果符合預算限制（超出預算 $\leq 5\%$ ）。
5. 從使用者送出查詢到收到回覆的整體回應時間 ≤ 30 秒（含 LLM 推論）。

4.2 非功能性目標

1. **隱私保護**：所有 LLM 推論在本地端執行（Ollama），使用者對話資料不傳送至外部雲端服務。
2. **可擴充性**：後端多餐廳管理架構支援動態新增、切換與刪除餐廳菜單，無需重啟服務。
3. **容錯性**：三層備援機制（CSS 選擇器 $\rightarrow$ JavaScript 注入 $\rightarrow$ 視覺模型）確保菜單取得的穩健性。

4.3 學術貢獻

本研究展示了以本地端 LLM 替代雲端 API 進行對話式推薦的可行性，並提出結合規則式關鍵字比對與 LLM 語意理解的混合式偏好抽取框架，可供後續研究作為基準參考。

5 研究流程規劃

本研究預計在一個學期（約 18 週）內完成，各階段工作如表 1 所示。

表 1: 研究甘特圖

工作項目	W1-2	W3-4	W5-6	W7-8	W9-10	W11-12	W13-14	W15-16	W17-18
文獻蒐集與整理	●	●							
環境建置	●	●							
爬蟲模組開發			●	●					
偏好理解模組開發				●	●				
推薦演算法實作					●	●			
前後端整合						●	●		
視覺備援模組							●	●	
測試與評估								●	●
報告撰寫與修訂								●	●

各主要里程碑如下：

- **第 6 週**：完成爬蟲模組，可成功爬取至少 3 間餐廳菜單。
- **第 10 週**：完成偏好抽取與推薦演算法，通過基本功能測試。
- **第 14 週**：完成前後端整合，系統可端對端運行。
- **第 16 週**：完成視覺備援模組測試，整合至主系統。
- **第 18 週**：完成期末報告與成果發表。

參考文獻

References

[1] Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., ... & Lample, G. (2023). *LLaMA: Open and Efficient Foundation Language Models*. arXiv preprint arXiv:2302.13971.

[2] Gemma Team. (2024). *Gemma: Open Models Based on Gemini Research and Technology*. arXiv preprint arXiv:2403.08295.

[3] Sun, Y., & Zhang, Y. (2018). Conversational Recommender System. In *Proceedings of the 41st International ACM SIGIR Conference on Research & Development in Information Retrieval* (pp. 235–244). ACM.

[4] Microsoft Corporation. (2024). *Playwright: Fast and Reliable End-to-End Testing for Modern Web Apps*. <https://playwright.dev>

[5] Chen, J., Huang, H., Liu, T., Su, J., & Chen, K. (2023). *OCR-Free Document Understanding Transformer*. In *European Conference on Computer Vision (ECCV 2022)*, Lecture Notes in Computer Science, vol. 13688, pp. 498–517. Springer.

[6] Ramírez, S. (2024). *FastAPI: Modern, Fast (High-Performance) Web Framework for Building APIs with Python 3.8+*. <https://fastapi.tiangolo.com>

- [7] Ollama Team. (2024). *Ollama: Get Up and Running with Large Language Models Locally*. <https://ollama.com>